

Proyecto de prueba

Este es el proyecto de prueba en el que trabajarás. Está dividido en varias secciones para que puedas elegir dónde tiene más sentido comenzar.

IaaS

Para esta sección, deberás crear un clúster de Fargate alojado en AWS mediante el servicio ECS y ECR. Para lograr esto, deberá proporcionar la siguiente lista de **requisitos**:

1. El aprovisionamiento **DEBE** gestionarse mediante la técnica IaaS. Nuestra herramienta preferida para realizar el trabajo es AWS CDK, pero puedes sugerir cualquier otra herramienta que consiga el resultado final. Si opta por utilizar otra herramienta, nos gustaría que nos proporcione los motivos (beneficios, desventajas) de la herramienta de su elección.
2. El clúster **DEBE** vivir y utilizar VPC y subredes personalizadas.
3. Las subredes del clúster y la VPC **DEBEN** de igual construirse con IaaS. Los valores de estas (CIDR, cantidad de sus redes públicas, privadas y aislada) deben proporcionarse como valores parametrizables (archivo de variables) o calcularse mediante cualquier otro método.
4. El clúster **DEBE** tener al menos dos aplicaciones corriendo (es de tu libre elección elegir estas aplicaciones), debes definir la mejor arquitectura que consideres para desplegar estas aplicaciones (un servicio con dos tasks, dos servicios cada uno con un task, u otra de tu preferencia) y los motivos (beneficios, desventajas) de tu elección.
5. La comunicación con las aplicaciones **DEBE** realizarse mediante una URL pública (Load Balancer, API Gateway u otro). El acceso a través de subredes privada NO DEBE estar habilitado.
6. El grupo de nodos o tasks **NO DEBE** tener IP públicas habilitadas de forma predeterminada.
7. Se **DEBE** poder acceder desde internet a las aplicaciones del clúster solo a través de HTTPS (puerto 443).
8. Desde IaaS se **DEBE** establecer al menos una capa de seguridad para garantizar en caso de requerirse, poder limitar el acceso a un listado de IPs públicas registradas en un archivo de parametrización.
9. Se **DEBE** garantizar alta disponibilidad de las aplicaciones y disponer de escenarios de fallas sobre la misma y/o de auto escalado ante escenario como crecimiento de la CPU.

Nota: Recuerde que todos estos puntos deben ser configurados con IaaS.

Solución propuesta:

Uso de Terraform para despliegues de IaaS

- Se propone utilizar **Terraform** para el despliegue de Infraestructura como código (IaaS). Motivo: **Terraform** tiene módulos certificados por AWS que pueden detallarse en el siguiente link: <https://registry.terraform.io/search/modules?namespace=terraform-aws-modules>

Módulos Terraform certificados por AWS

- El uso de **módulos Terraform certificados por AWS** permite simplificar la administración de la IaaS, mayor sencillez en la administración, evitar errores de código y trabajar a mayor alto nivel en los despliegues de infraestructura.

Servidor Terraform (virtual o físico)

- En general la metodología planteada para solventar esta prueba está basada en los siguientes hitos:
 - Creación de servidor para Terraform (para despliegues de IaaS). Nota 1

- Instalación de AWS CLI (en el servidor Terraform)
- Conectividad de Terraform al tenance AWS CLI (ID, key_pair, zone).
- Instalar los módulos Terraform AWS a usar en esta prueba:
 - i. Para creación/edición de redes **VPC** la propuesta: terraform-aws-modules/vpc.
URL:
<https://registry.terraform.io/modules/terraform-aws-modules/vpc/aws/latest>
 - ii. Para creación/edición del Cluster **EKS** la propuesta: terraform-aws-modules/eks.
URL:
<https://registry.terraform.io/modules/terraform-aws-modules/eks/aws/latest>
 - iii. Para usuarios y permisos IAM la propuesta es utilizar:
URL:
<https://registry.terraform.io/modules/terraform-aws-modules/iam/aws/latest>
 - iv. La configuración en este directorio crea un grupo de nodos administrados por EKS junto con un rol de IAM, un grupo de seguridad y una plantilla de lanzamiento.
URL usar:
<https://registry.terraform.io/modules/terraform-aws-modules/eks/aws/latest/submodules/eks-managed-node-group>

Requisitos (puntos 1-9)

1. Gestion mediante IaC (Terraform versus CDK).
 - Beneficios/Ventajas de Terraform: Es el estándar de facto para la industria de IaC. Permite arquitecturas híbridas (cloud, on-premise), permite multiprovedores en un solo sistema (AWS, Azure, Google, IBMcloud, DigitalOcean, OpenShift, etc).
 - Beneficios/Ventajas AWS CDK: solo para AWS, excelente documentación y soporte técnico.
 - Terraform permite manejo de nubes híbridas, ambientes cloud y on-premise en una sola plataforma. CDK es único para AWS. Permite independencia de proveedores múltiples.
 - Desventajas: Terraform es open-source a pesar de su excelente documentación, no hay un proveedor atrás para funciones de escalación, consulta, manejo de casos, etc.
2. Debe vivir y utilizar VPC y subredes propias.
 - a. El módulo AWS VPC para kubernetes permite definir-crear todos los componentes de red. Ejemplo:

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"

  name = "my-vpc"
  cidr = "10.0.0.0/16"

  azs          = ["eu-west-1a", "eu-west-1b", "eu-west-1c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24", "10.0.103.0/24"]

  enable_nat_gateway = true
  enable_vpn_gateway = true

  tags = {
    Terraform = "true"
    Environment = "dev"
  }
}
```

- b. Observe en el ejemplo: múltiples zonas, redes públicas, redes privadas para "my-vpc"

3. Las VPC y todos sus elementos deben configurarse bajo laaC
 - a. La propuesta está basada en el módulo AWS VPC que es únicamente para redes.
 - b. El servidor de Terraform dispondrá de un filesystem /terraform/aws/vpc en donde estarán todas las configuraciones (tf) para crear y modificar los componentes de red
 - c. En la propuesta el usuario con rol full_admin podrá ingresar al filesystem para configurar, crear, borrar cualquier asociado al modulo AWS VPC
 - d. Ejemplos de configuración de módulo AWS VPC en <https://registry.terraform.io/modules/terraform-aws-modules/vpc/aws/latest>
4. El Clúster debe disponer de 2 aplicaciones. Luego de la construcción secuencial de:
 - a. Creación de VPC y sus elementos (módulo AWS VPC) en /terraform/aws/vpc
 - b. Creación de Cluster EKS (módulo AWS EKS) en /terraform/aws/eks
 - c. Se plantea 2 aplicaciones de prueba que se desarrollan en la sección de cicd.
 - i. Example_01_Simple_Web_App.yaml
 - ii. Example_02_Microservice_App.yaml
 - d. Los manifiestos del punto anterior (yaml) crean 2 ejemplos en EKS fargate. Un servidor web simple (basado en nginx) y un microservicio (my-frontend-app)
5. Solo comunicación por URL Publica (privadas deshabilitadas)
 - a. El módulo AWS EKS permite parametrizar la creación de accesos privado/publico
 - b. Colocar en False la variable: cluster_endpoint_private_access
 - c. Colocar en True la variable: cluster_endpoint_private_access

<code>cluster_endpoint_private_access</code>	Indicates whether or not the Amazon EKS private API server endpoint is enabled
<code>cluster_endpoint_public_access</code>	Indicates whether or not the Amazon EKS public API server endpoint is enabled
<code>cluster_endpoint_public_access_cidrs</code>	List of CIDR blocks which can access the Amazon EKS public API server endpoint

- d. Ejemplo manifiesto tf (parcial)

```
source = "terraform-aws-modules/eks/aws"
version = "~> 18.0"

cluster_name = "vcc-eks-tf"
cluster_version = "1.22"

cluster_endpoint_private_access = true
cluster_endpoint_public_access = true
```

- e. Mayor información de parámetros : https://registry.terraform.io/modules/terraform-aws-modules/eks/aws/latest#output_cluster_endpoint

6. Nodos/Tasks solo por redes Publicas
 - a. Detalle en el link siguiente la parametrización: https://github.com/terraform-aws-modules/terraform-aws-eks/blob/master/docs/network_connectivity.md#public-endpoint-w-restricted-cidrs

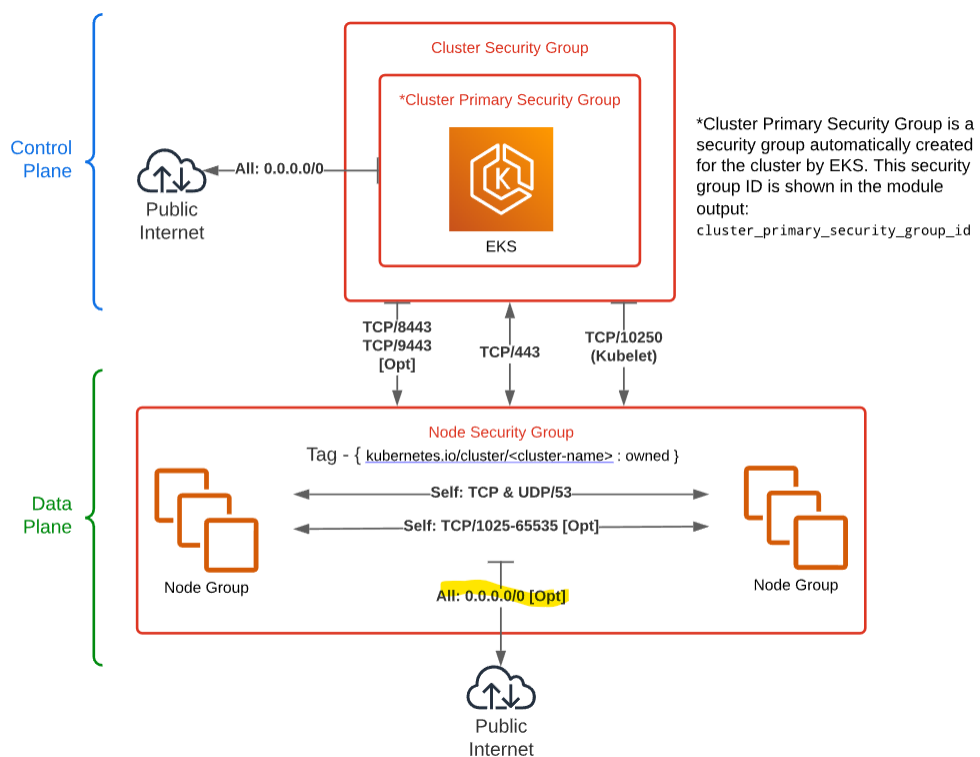
```

...
# Extend cluster security group rules
cluster_security_group_additional_rules = {
    egress_nodes_ephemeral_ports_tcp = {
        description = "To node 1025-65535"
        protocol     = "tcp"
        from_port    = 1025
        to_port      = 65535
        type         = "egress"
        source_node_security_group = true
    }
}

# Extend node-to-node security group rules
node_security_group_additional_rules = {
    ingress_self_all = {
        description = "Node to node all ports/protocols"
        protocol     = "-1"
        from_port    = 0
        to_port      = 0
        type         = "ingress"
        self         = true
    }
    egress_all = {
        description = "Node all egress"
        protocol     = "-1"
        from_port    = 0
        to_port      = 0
        type         = "egress"
        cidr_blocks  = ["0.0.0.0/0"]
        ipv6_cidr_blocks = [ ":::/0"]
    }
}

```

Esquema general:



7. Acceso internet por 443. En el punto anterior limitar `from_port = 443` & `to_port = 443`

8. Seguridad restricción en las IPs públicas.

a. Para limitar acceso público configurar `cluster_endpoint_public_access_cidrs`

- b. Este recurso permite: “List of CIDR blocks which can access the Amazon EKS public API server endpoint”
 - c. https://github.com/terraform-aws-modules/terraform-aws-eks/blob/master/docs/network_connectivity.md#public-endpoint-w-restricted-cidrs
9. Alta disponibilidad (no desarrollado por tiempo límite de 24H, pendiente). Desarrollar:
- a. Nodos de clúster EKS separados en AZ distintas
 - b. Balanceadores distribuidos entre varias AZ
 - c. Repositorios de objetos en S3 replicados
 - d. Despliegues balanceados
 - e. Revisar Karpenter (herramienta de aprovisionamiento de nodos que ayuda a optimizar el escalado del clúster y el uso de recursos).
 - f. Ref: <https://docs.aws.amazon.com/eks/latest/userguide/autoscaling.html>